Augmenting data structures solutions

14-E01 Select and size

Derivation and check

x has local rank5+1=6. Since8>6, search right for local rank8−6=2. Thus select(right,2). Reaching NIL would signal a k outside the subtree range and must be rejected.

14-E02 Updating size on rotation

Derivation and check

After rotation, first x.size=x.left.size+beta.size+1; then y.size=x.size+y.right.size+1. Computing y first reads old x.size. Other ancestor sizes are unchanged by rotation, but root and parent wiring must still be correct.

14-E03 Interval overlap search

Derivation and check

Query[24,26] overlaps root directly because20≤26 and30≥24; return it. Query[31,33] misses root and left.max25<31 excludes the left side, so go right, whose max34 allows a possible match. In general, if x.low>q.high, any candidate must be left, and left.max≥q.low is needed to keep that side feasible. Root interval alone cannot establish global absence.

14-E04 Select and rank in the same tree

Derivation and check

Select starts at20 with r4,k7, goes40 with k3; at40 r5 leads left to30 with k3; at30 r3 returns30. Rank27 starts1, adds1 on right ascent to25, adds nothing on left ascents to30/40, and adds size10+1=4 on right ascent to20, giving6. After deletion30 loses25 but retains27, so size30=3,size40=5,new25.size=3+5+1=9. Do not retain moved25’s old size2; recompute from final children, descendants before ancestors.

Augmenting data structures solutions (continued)

14-E05 Local formulas versus global-rank maintenance

Derivation and check

A new minimum raises every old global rank by1, requiring n explicit rewrites. Subtree size changes only along the new leaf’ s ancestor path, O(log n) in a red-black tree. A child summary such as height can change at the local root after rotation and propagate to ancestors. Size/max aggregate contents, and the whole local subtree retains the same contents, so two pivot-node recomputations leave ancestors unchanged. This additional property does not hold for every child formula.

14-E06 A feasible maximum does not guarantee overlap

Derivation and check

left.max40≥5 holds, so search descends to[10,40]. It lies wholly right of the query since10>6; no children remain and search returns NIL. Maximum-attaining j=[10,40] has j.high≥5 but misses, forcing j.low10>q.high6. All right lows≥root.low20≥j.low10>6 also miss; root misses too. The feasible max test only prevents pruning left on small highs; it does not promise a match. Correctness uses global low order and the invariant.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Augmenting data structures solutions (continued)

Verification code appendix: augmentation_checks.py

```
"""Augmentation behavior on an ordinary BST; logarithmic height relies on the separate RB
proof."""
from pathlib import Path
import sys
import random
sys.path.insert(0,str(Path(__file__).resolve().parents[2]/"ch12-binary-search-
trees/examples"))
from bst_checks import Tree


def size(node):
    return 0 if node is None else node.size


def maximum(node):
    return float('-inf') if node is None else node.maximum


def low(node):
    return node.key[0] if isinstance(node.key,tuple) else node.key


def pull(node):
    node.size=1+size(node.left)+size(node.right)
    node.maximum=max(node.high,maximum(node.left),maximum(node.right))


def refresh_up(node):
    while node is not None:
        pull(node)
        node=node.parent
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Augmenting data structures solutions (continued)

```
class AugTree(Tree):
    def insert(self,key,high=None):
        node=super().insert(key)
        node.high=low(node) if high is None else high
        refresh_up(node)
        return node
    def delete(self,z):
        start=z.parent
        if z.left is not None and z.right is not None:
            y=z.right
            while y.left is not None:
                y=y.left
            start=y if y.parent is z else y.parent
        super().delete(z)
        refresh_up(start)
    def rotate_left(self,x):
        y=x.right
        assert y is not None
        p=x.parent
        x.right=y.left
        if y.left is not None:
            y.left.parent=x
        y.parent=p
        if p is None:
            self.root=y
        elif p.left is x:
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Augmenting data structures solutions (continued)

```
            p.left=y
        else:
            p.right=y
        y.left=x
        x.parent=y
        pull(x)
        pull(y)
        return y
    def rotate_right(self,y):
        x=y.left
        assert x is not None
        p=y.parent
        y.left=x.right
        if x.right is not None:
            x.right.parent=y
        x.parent=p
        if p is None:
            self.root=x
        elif p.left is y:
            p.left=x
        else:
            p.right=x
        x.right=y
        y.parent=x
        pull(y)
        pull(x)
        return x
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Augmenting data structures solutions (continued)

```
def select(node,k):
    if not 1<=k<=size(node):
        raise ValueError("rank outside subtree")
    while node is not None:
        r=size(node.left)+1
        if k==r:
            return node
        if k<r:
            node=node.left
        else:
            k-=r
            node=node.right


def rank(node):
    result=size(node.left)+1
    while node.parent is not None:
        if node is node.parent.right:
            result+=size(node.parent.left)+1
        node=node.parent
    return result


def overlap_search(tree,a,b):
    node=tree.root
    path=[]
    while node is not None and not (low(node)<=b and a<=node.high):
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Augmenting data structures solutions (continued)

```
        path.append(low(node))
        if node.left is not None and node.left.maximum>=a:
            node=node.left
        else:
            node=node.right
    if node is not None:
        path.append(low(node))
    return node,path


def validate(tree,records):
    seen=set()
    def visit(node,parent,lo,hi):
        if node is None:
            return 0,float('-inf'),[]
        assert node not in seen and node.parent is parent
        assert node.high==records[node.key]
        assert (lo is None or lo<node.key) and (hi is None or node.key<hi)
        seen.add(node)
        ls,lmax,left=visit(node.left,node,lo,node.key)
        rs,rmax,right=visit(node.right,node,node.key,hi)
        assert node.size==ls+rs+1
        assert node.maximum==max(node.high,lmax,rmax)
        return node.size,node.maximum,left+[node.key]+right
    _,_,ordered=visit(tree.root,None,None,None)
    assert ordered==sorted(records)
    for i,key in enumerate(ordered,1):
        node=tree.search(key)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Augmenting data structures solutions (continued)

```
        assert select(tree.root,i) is node and rank(node)==i


INTERVALS=[(12,17),(5,7),(20,24),(2,4),(8,15),(16,19),(25,27)]


def main():
    tree=AugTree([20,10,40,5,15,30,50,25,35,27])
    assert select(tree.root,7).key==30 and rank(tree.search(27))==6
    tree.delete(tree.search(20))
    assert tree.search(30).size==3 and tree.search(40).size==5 and tree.root.size==9
    rng=random.Random(20261024)
    tree=AugTree()
    records={}
    rotations=0
    for step in range(1500):
        key=rng.randrange(50)
        if rng.randrange(2):
            high=key+rng.randrange(20)
            tree.insert(key,high)
            records[key]=high
        elif key in records:
            tree.delete(tree.search(key))
            del records[key]
        validate(tree,records)
        if records and step%7==0:
            candidates=[tree.search(key) for key in records if tree.search(key).right is not
None]
            if candidates:
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Augmenting data structures solutions (continued)

```
            x=rng.choice(candidates)
            y=tree.rotate_left(x)
            validate(tree,records)
            tree.rotate_right(y)
            validate(tree,records)
            rotations+=2
intervals=AugTree()
for identity,(a,b) in enumerate(INTERVALS):
    intervals.insert((a,identity),b)
assert overlap_search(intervals,9,10)[1]==[12,5,8]
assert overlap_search(intervals,18,18)[1]==[12,20,16]
assert overlap_search(intervals,28,29)==(None,[12,20,25])
assert overlap_search(intervals,4,5)[0] is not None
records={}
tree=AugTree()
queries=0
for step in range(220):
    identity=rng.randrange(16)
    previous=next((key for key in records if key[1]==identity),None)
    if previous is not None:
        tree.delete(tree.search(previous))
        del records[previous]
    if rng.randrange(3):
        a=rng.randrange(25)
        b=a+rng.randrange(8)
        tree.insert((a,identity),b)
        records[(a,identity)]=b
    validate(tree,records)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Augmenting data structures solutions (continued)

```
        if records and step%5==0:
            candidates=[tree.search(key) for key in records if tree.search(key).right is not
None]
            if candidates:
                tree.rotate_left(rng.choice(candidates))
                validate(tree,records)
                rotations+=1
        for a in range(-1,33):
            for b in range(a,33):
                found,_=overlap_search(tree,a,b)
                expected=any(key[0]<=b and a<=high for key,high in records.items())
                assert (found is not None)==expected
                if found is not None:
                    assert low(found)<=b and a<=found.high
                queries+=1
    rotation_example=AugTree()
    for a,b in [(12,17),(5,7),(20,24),(8,15),(16,19),(25,27)]:
        rotation_example.insert(a,b)
    x=rotation_example.root
    assert x.maximum==27
    y=rotation_example.rotate_left(x)
    assert x.maximum==19 and y.maximum==27
    print(f"PASS augmented_mixed_updates=1500 actual_rotations={rotations}
rank_select_all_nodes=True interval_queries_against_exhaustive_scan={queries}
closed_endpoints_and_duplicate_lows=True delete_size_trace=(3,5,9) rotated_max=(19,27)")


if __name__=='__main__':
    main()
```

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study